

HOUSE PRICE PREDICTION USING LINEAR REGRESSION

Internship Project Report

Submitted By: Sai Deva Harsha

Company: GEONIXA

Problem Statement

The real estate market is influenced by multiple factors such as property area, number of bedrooms, bathrooms, parking availability, property age, location rating, and distance from the city center. Estimating the selling price of a house manually can be difficult and inconsistent. The objective of this project is to develop a House Price Prediction system using Linear Regression that can estimate the selling price of a house based on these factors. The system should help buyers, sellers, and real-estate agencies make data-driven decisions by providing accurate price predictions.

Introduction

Machine Learning is widely used in real-world applications for prediction and analysis. House price prediction is one of the important applications of machine learning in the real estate industry.

In this project, a Linear Regression model was developed to predict house prices using different housing features. The project includes data preprocessing, visualization, model training, prediction, and evaluation.

Dataset Used

The dataset used for this project is House Price Prediction Dataset.

Features included in the dataset:

- Area_sqft
- Bedrooms
- Bathrooms
- Parking
- House_Age
- Location_Rating
- Distance_City_km
- Price

Target Variable:

- Price

Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

Algorithm Used

Linear Regression is a supervised machine learning algorithm used to predict continuous values.

The Linear Regression model learns relationships between independent variables and the target variable and predicts house prices accurately.

Steps Performed

1. Imported required libraries
2. Loaded dataset using Pandas
3. Checked dataset information
4. Handled missing values
5. Performed data visualization
6. Split dataset into training and testing data
7. Trained Linear Regression model
8. Predicted house prices
9. Evaluated model performance
10. Predicted price for a new house

Data Visualization

The following visualization techniques were used:

- Correlation Heatmap
- Pairplot
- Histogram

- Boxplot
- Scatter Plot
- Residual Plot

Model Training

The dataset was divided into training data and testing data.

Training Data : 80% Testing

Data : 20%

The Linear Regression model was trained using the training dataset.

Model Evaluation

The model performance was evaluated using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Results

The Linear Regression model successfully predicted house prices using features such as area, bedrooms, bathrooms, parking, house age, location rating, and distance from the city center.

The evaluation metrics and visualization graphs indicated that the model performed well on the given dataset. The Actual vs Predicted graph showed that the predicted values were close to the actual values.

What I Learned

Through this project, I learned:

- Basics of Machine Learning
- Working with datasets using Pandas
- Data preprocessing techniques
- Data visualization using Matplotlib and Seaborn
- Linear Regression algorithm
- Model training and prediction
- Model evaluation techniques
- Real-world application of machine learning

Advantages

- Fast house price prediction
- Helps buyers and sellers
- Data-driven decision making
- Easy to implement

Future Enhancements

- Use advanced algorithms like Random Forest and XGBoost
- Build a web application
- Add real-time property data
- Improve prediction accuracy

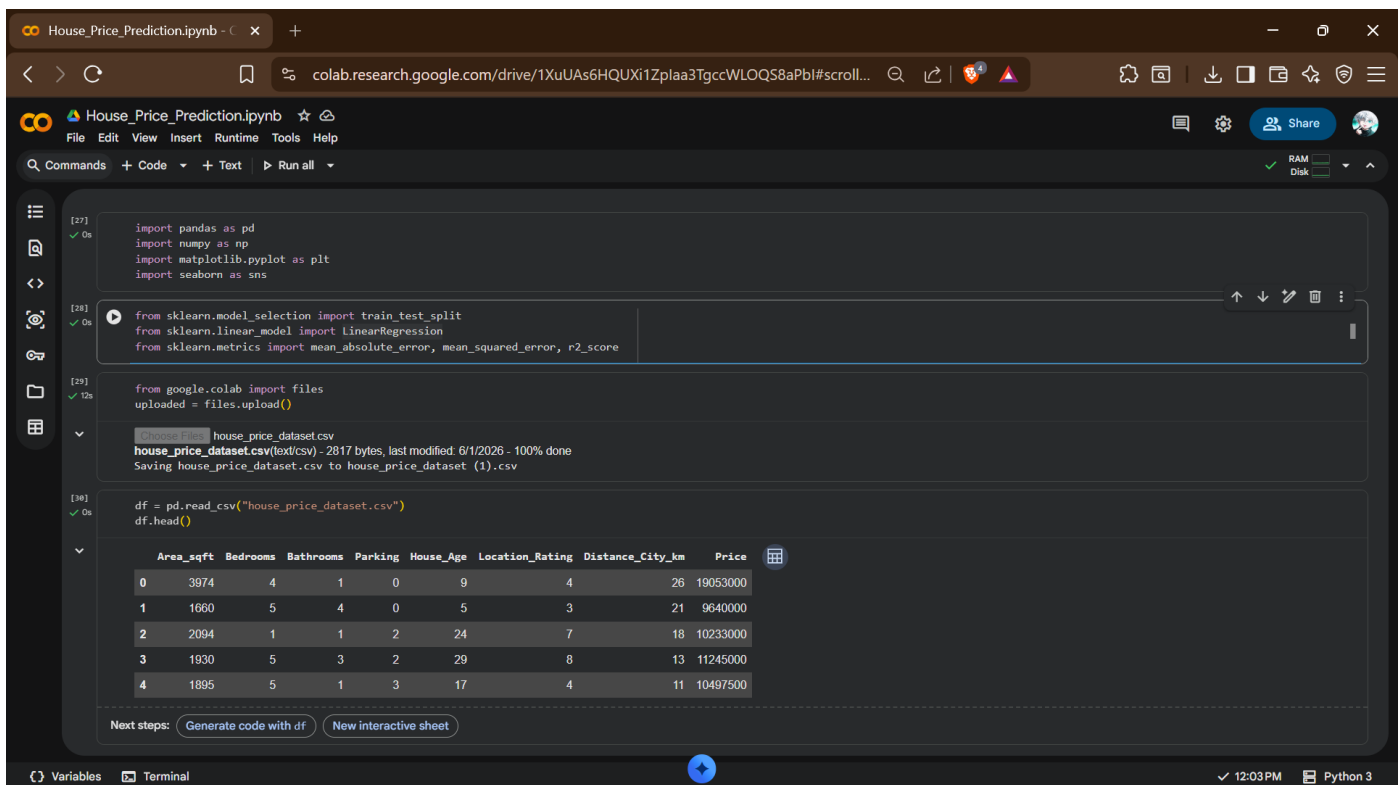
Conclusion

The House Price Prediction system using Linear Regression was successfully developed and implemented. The project analyzed housing data, visualized feature relationships, trained a machine learning model, and predicted house prices effectively.

This project demonstrated how machine learning techniques can solve real-world problems in the real estate industry.

The project also helped in understanding the complete machine learning workflow from data preprocessing to model evaluation and prediction.

Outputs



The screenshot displays a Google Colab notebook interface with the following content:

```
[27] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

[28] from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

[29] from google.colab import files
uploaded = files.upload()

Choose File house_price_dataset.csv
house_price_dataset.csv(text/csv) - 2817 bytes, last modified: 6/1/2026 - 100% done
Saving house_price_dataset.csv to house_price_dataset (1).csv

[30] df = pd.read_csv("house_price_dataset.csv")
df.head()
```

	Area_sqft	Bedrooms	Bathrooms	Parking	House_Age	Location_Rating	Distance_City_km	Price
0	3974	4	1	0	9	4	26	19053000
1	1660	5	4	0	5	3	21	9640000
2	2094	1	1	2	24	7	18	10233000
3	1930	5	3	2	29	8	13	11245000
4	1895	5	1	3	17	4	11	10497500

Next steps: [Generate code with df](#) [New interactive sheet](#)

Variables Terminal 12:03 PM Python 3

House_Price_Prediction.ipynb - C x +

colab.research.google.com/drive/1XuUAs6HQUXi1Zplaa3TgccWLOQS8aPbl#scroll...

House_Price_Prediction.ipynb ☆

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

[31] ✓ 0s

```
print("Shape of Dataset:")
print(df.shape)

print("\nColumns:")
print(df.columns)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())
```

Shape of Dataset:
(100, 8)

Columns:
Index(['Area_sqft', 'Bedrooms', 'Bathrooms', 'Parking', 'House_Age',
 'Location_Rating', 'Distance_City_km', 'Price'],
 dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 8 columns):
Column Non-Null Count Dtype
--- ---
0 Area_sqft 100 non-null int64
1 Bedrooms 100 non-null int64
2 Bathrooms 100 non-null int64
3 Parking 100 non-null int64
4 House_Age 100 non-null int64
5 Location_Rating 100 non-null int64
6 Distance_City_km 100 non-null int64
7 Price 100 non-null int64
dtypes: int64(8)
memory usage: 6.4 KB
None

Variables Terminal

12:03 PM Python 3

House_Price_Prediction.ipynb - C x +

colab.research.google.com/drive/1XuUAs6HQUXi1Zplaa3TgccWLOQS8aPbl#scroll...

House_Price_Prediction.ipynb ☆

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

None

Statistical Summary:

	Area_sqft	Bedrooms	Bathrooms	Parking	House_Age
count	100.000000	100.000000	100.000000	100.000000	100.000000
mean	2472.660000	2.910000	2.680000	1.650000	15.730000
std	918.624094	1.464013	1.144861	1.192358	8.819005
min	821.000000	1.000000	1.000000	0.000000	1.000000
25%	1735.750000	1.000000	2.000000	1.000000	8.000000
50%	2382.000000	3.000000	3.000000	2.000000	16.500000
75%	3233.500000	4.000000	4.000000	3.000000	23.000000
max	3974.000000	5.000000	4.000000	3.000000	29.000000

Location_Rating Distance_City_km Price

	Location_Rating	Distance_City_km	Price
count	100.000000	100.000000	1.000000e+02
mean	5.500000	15.960000	1.289317e+07
std	3.125328	7.825005	4.125033e+06
min	1.000000	3.000000	4.947000e+06
25%	3.000000	9.750000	9.635125e+06
50%	5.000000	16.500000	1.255750e+07
75%	8.000000	23.000000	1.648800e+07
max	10.000000	29.000000	2.077250e+07

[32] ✓ 0s

```
print(df.isnull().sum())
```

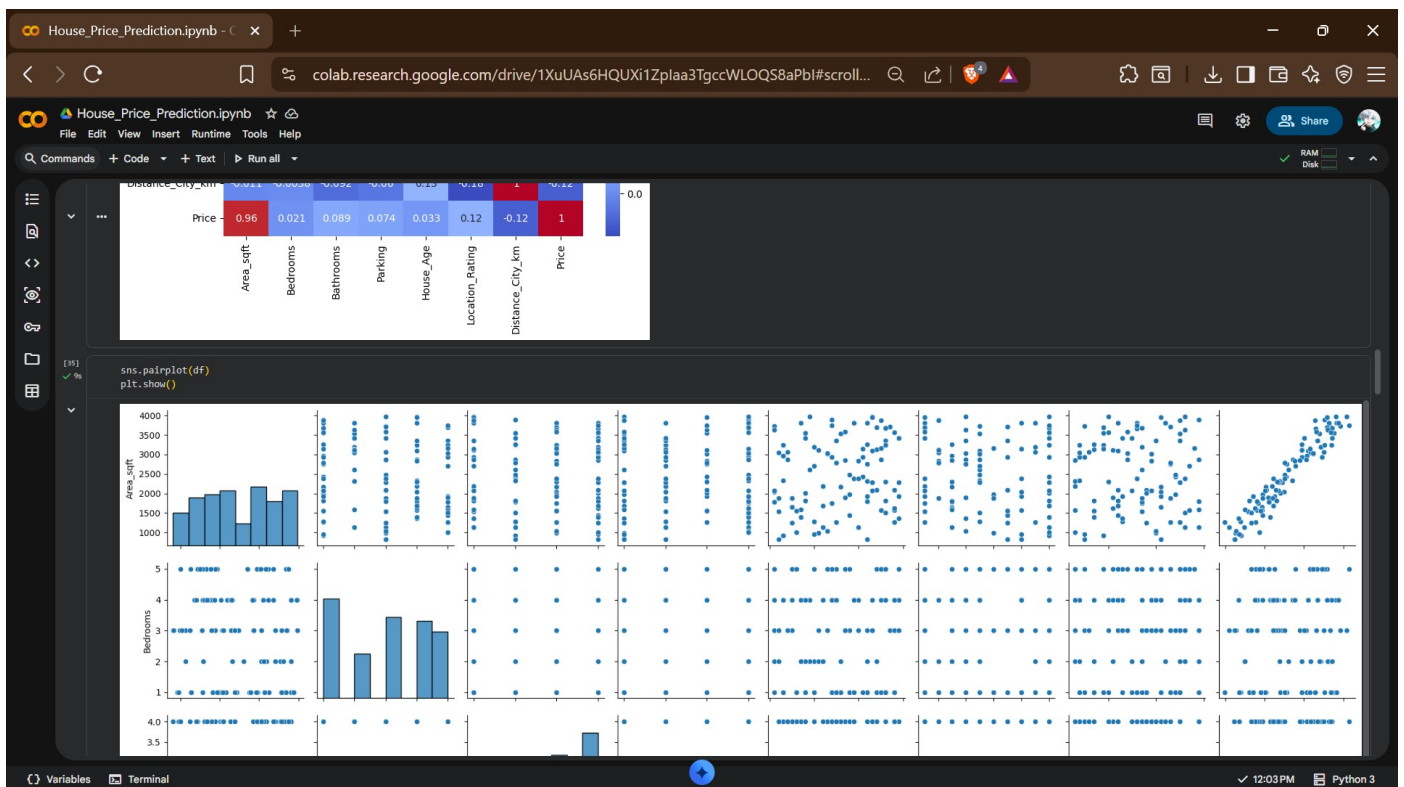
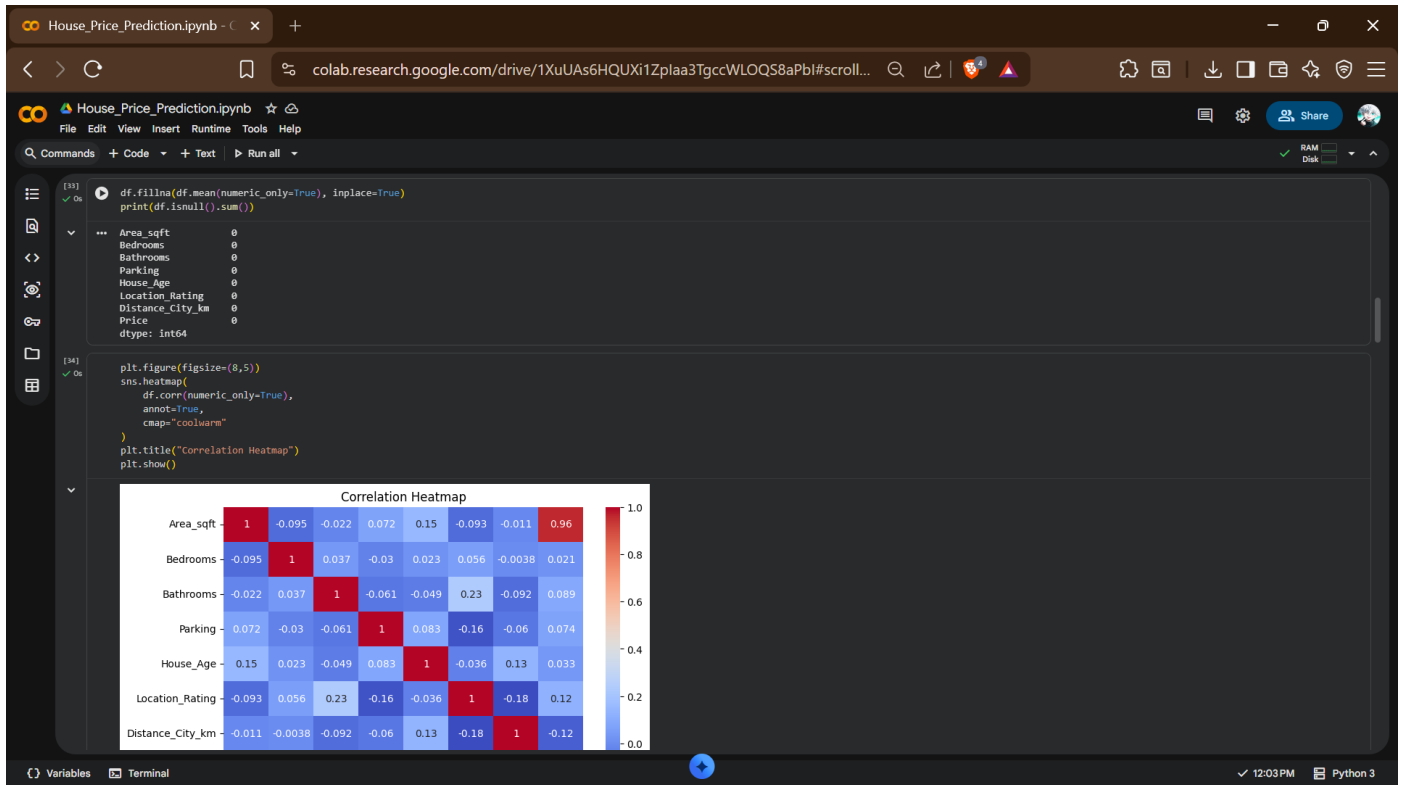
Area_sqft 0
Bedrooms 0
Bathrooms 0
Parking 0
House_Age 0
Location_Rating 0
Distance_City_km 0
Price 0
dtype: int64

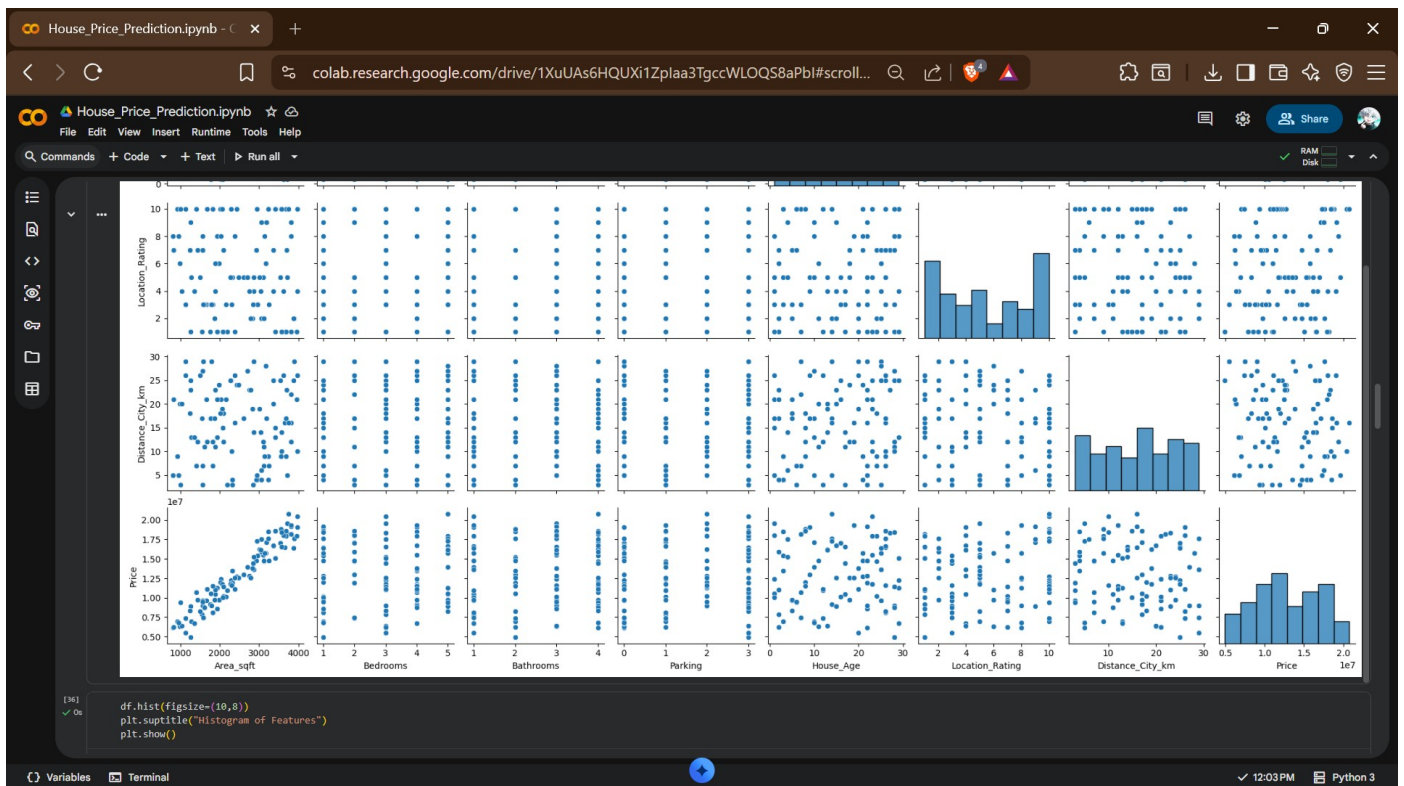
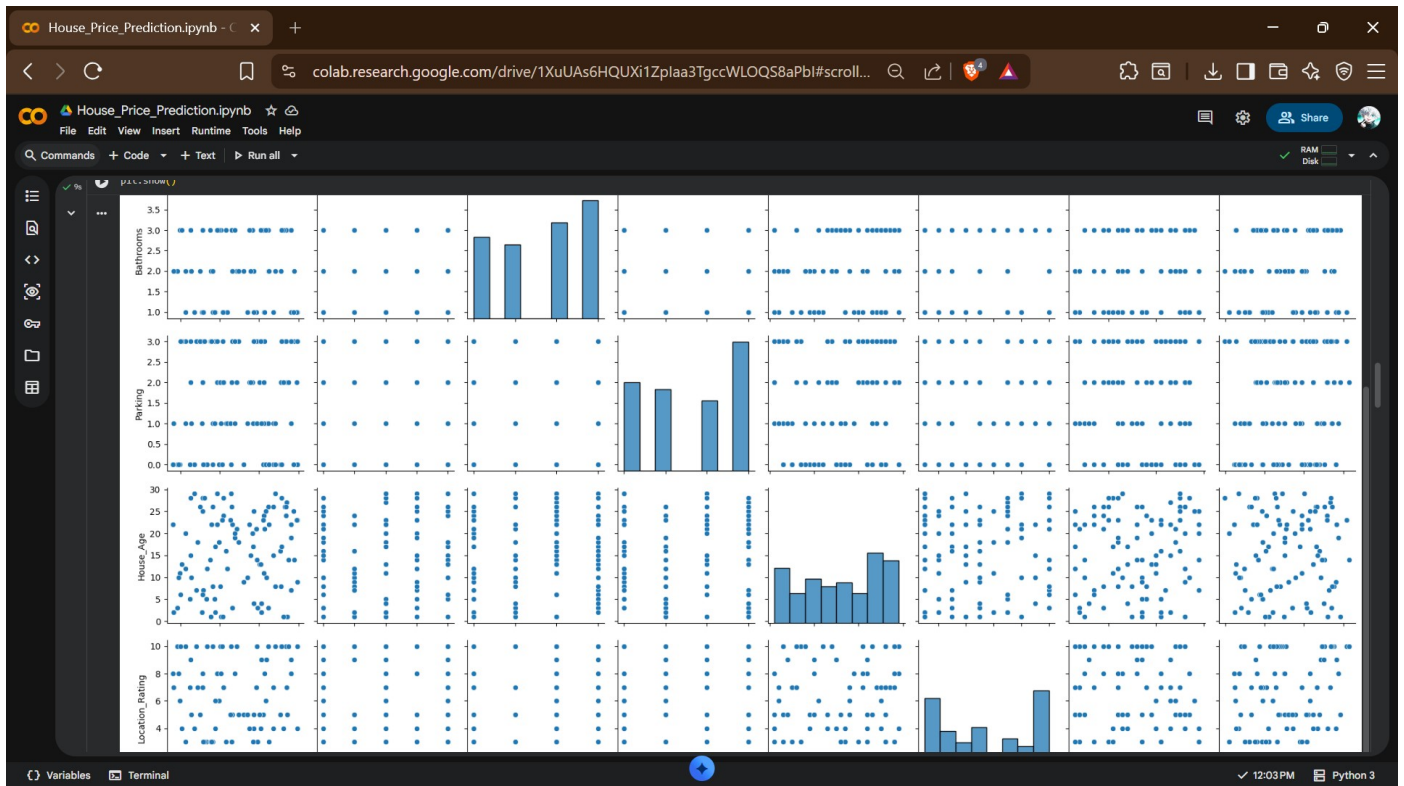
[33] ✓ 0s

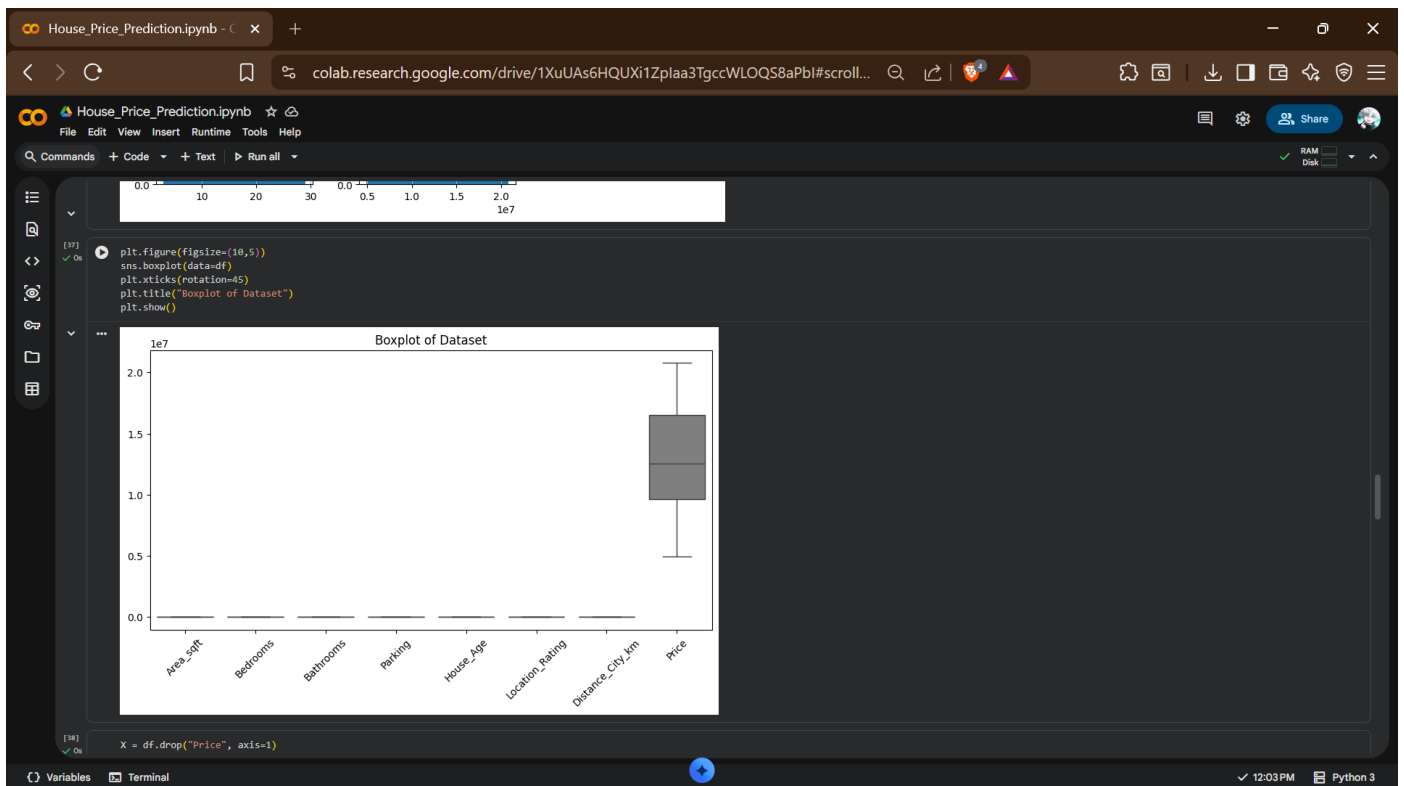
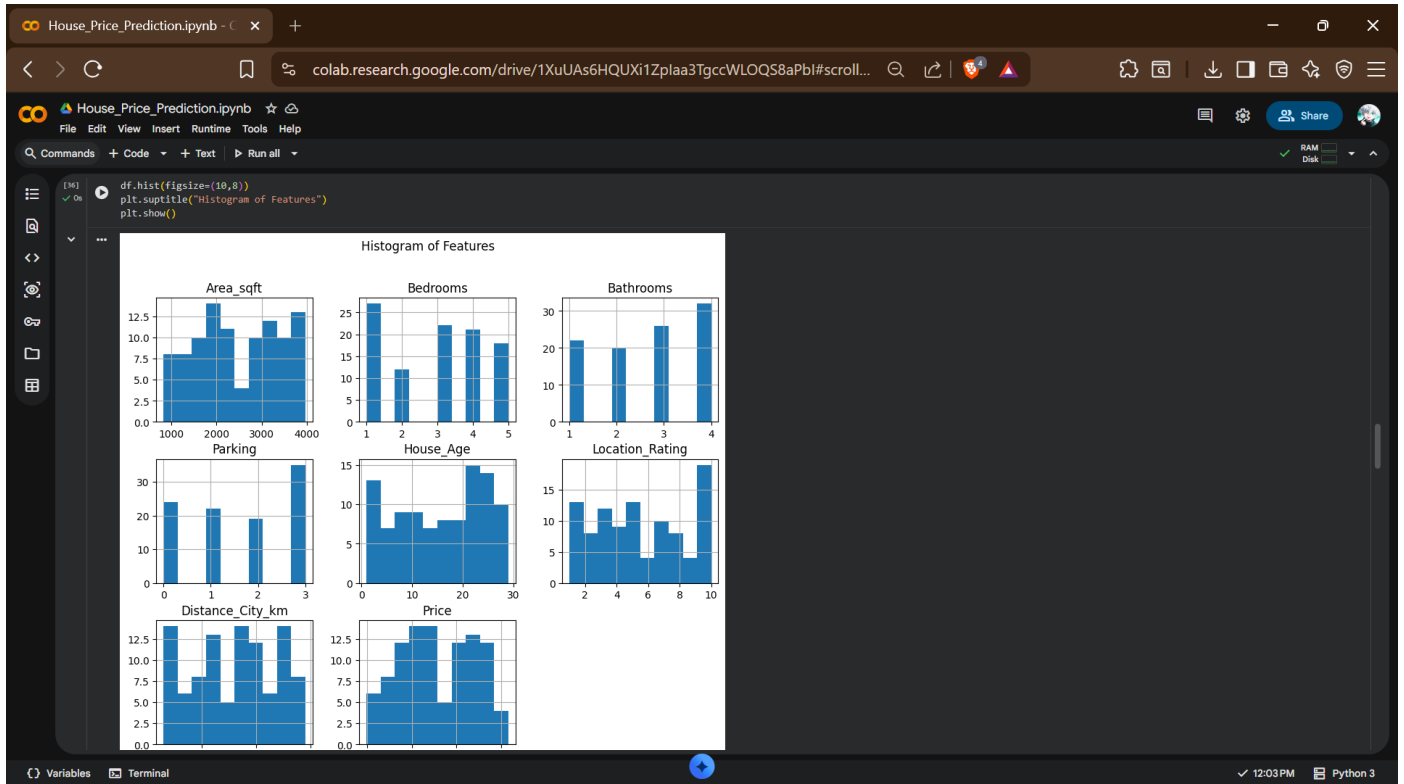
```
df.fillna(df.mean(numeric_only=True), inplace=True)
print(df.isnull().sum())
```

Variables Terminal

12:03 PM Python 3







House_Price_Prediction.ipynb - C x +

colab.research.google.com/drive/1XuUAs6HQUXi1Zplaa3TgccWLOQS8aPbl#scroll...

House_Price_Prediction.ipynb ☆

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

```
[38] ✓ On X = df.drop("Price", axis=1)
y = df["Price"]
print(X.head())
print(y.head())
```

	Area_sqft	Bedrooms	Bathrooms	Parking	House_Age	Location_Rating	
0	3974	4	1	0	9	4	
1	1660	5	4	0	5	3	
2	2094	1	1	2	24	7	
3	1930	5	3	2	29	8	
4	1895	5	1	3	17	4	

```
Distance_City_km
0      26
1      21
2      18
3      13
4      11
0  19053000
1  9640000
2  10233000
3  11245000
4  10407500
Name: Price, dtype: int64
```

```
[49] ✓ On X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
[50] ✓ On model = LinearRegression()
model.fit(X_train, y_train)
print("Model Trained Successfully")
```

Model Trained Successfully

```
[51] ✓ On y_pred = model.predict(X_test)
print(y_pred)
```

Variables Terminal

✓ 12:03 PM Python 3

House_Price_Prediction.ipynb - C x +

colab.research.google.com/drive/1XuUAs6HQUXi1Zplaa3TgccWLOQS8aPbl#scroll...

House_Price_Prediction.ipynb ☆

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

RAM Disk

```
[51] ✓ On y_pred = model.predict(X_test)
print(y_pred)
```

```
[52] ✓ On mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 Score :", r2)
```

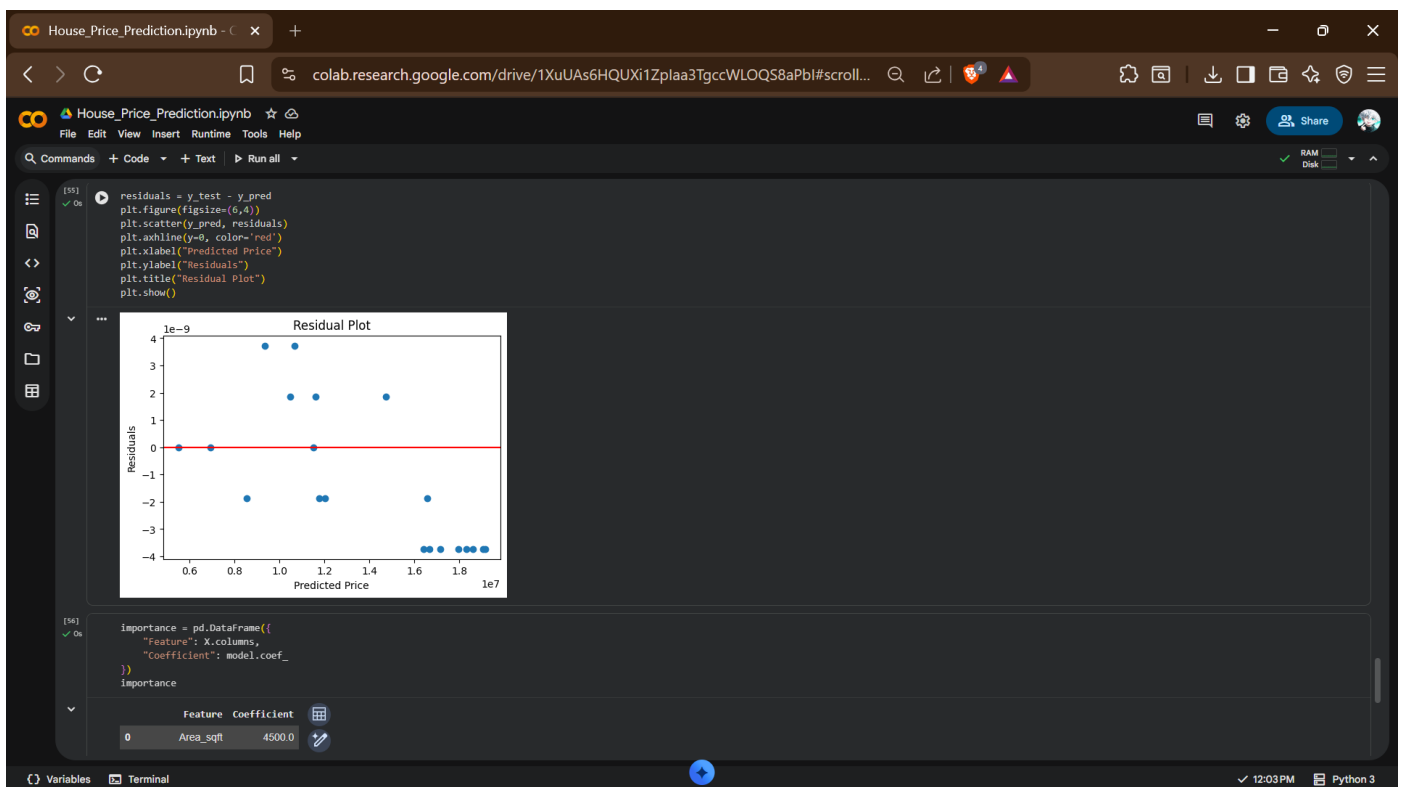
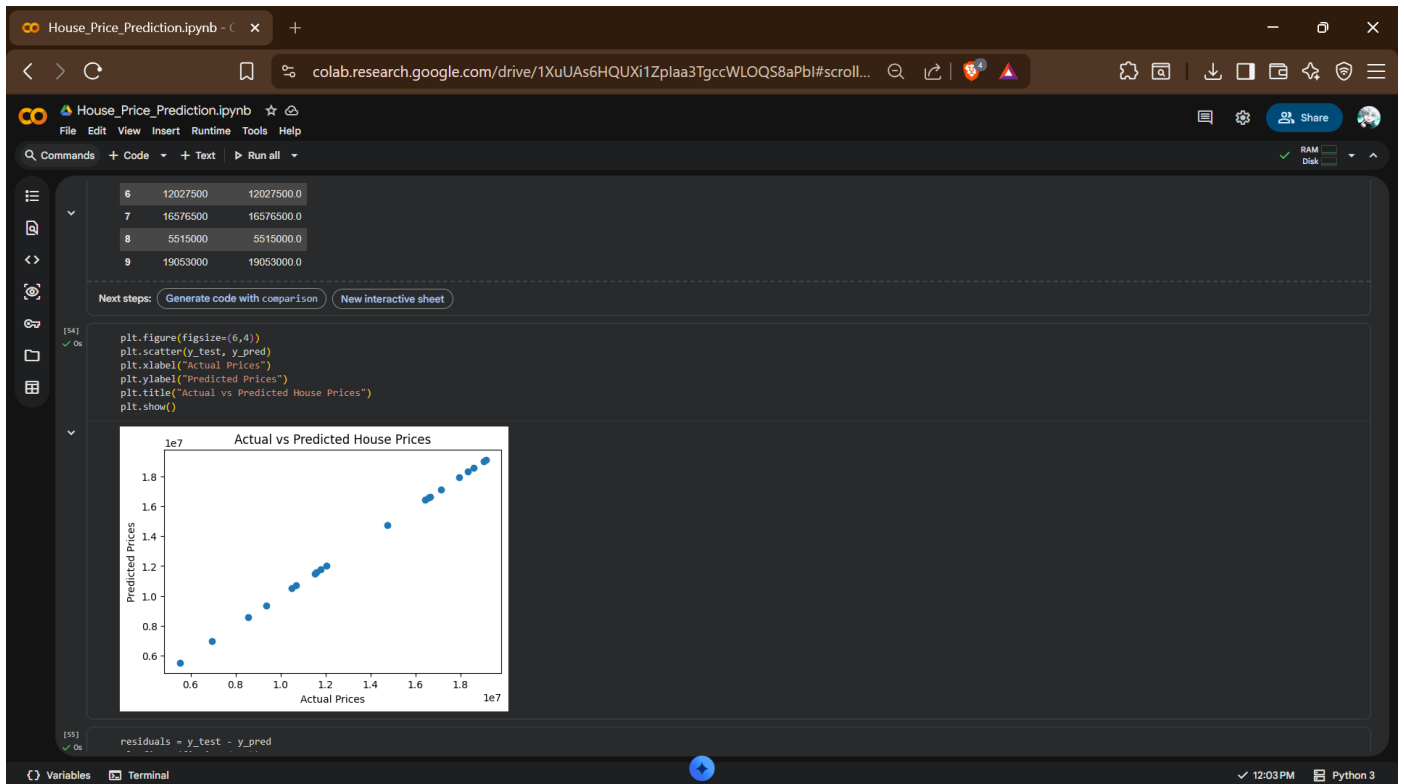
MAE : 2.514570951461792e-09
MSE : 8.153200337090994e-18
RMSE : 2.855380944303403e-09
R2 Score : 1.0

```
[53] ✓ On comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})
comparison.head(10)
```

	Actual Price	Predicted Price
0	9364500	9364500.0
1	10690000	10690000.0
2	17954000	17954000.0
3	18595500	18595500.0
4	11781500	11781500.0
5	19142500	19142500.0
6	12027500	12027500.0

Variables Terminal

✓ 12:03 PM Python 3



House_Price_Prediction.ipynb

colab.research.google.com/drive/1XuUAs6HQUXi1Zplaa3TgccWLOQS8aPbl#scroll...

House_Price_Prediction.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM

Disk

[56] 0s

Importance = pd.DataFrame({
 "Feature": X.columns,
 "Coefficient": model.coef_
})
Importance

	Feature	Coefficient
0	Area_sqft	45000.0
1	Bedrooms	300000.0
2	Bathrooms	200000.0
3	Parking	150000.0
4	House_Age	-50000.0
5	Location_Rating	250000.0
6	Distance_City_km	-30000.0

Next steps:

Generate code with Importance

New interactive sheet

[57] 0s

new_house = X.iloc[[0]].values
prediction = model.predict(new_house)
print("Predicted House Price :", prediction[0])

...

Predicted House Price : 19053000.000000004
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
 warnings.warn(

Variables

Terminal

12:03 PM

Python 3